International Journal of Cognitive Computing in Engineering 7 (2026) 145–154

Contents lists available at ScienceDirect

International Journal of Cognitive Computing in Engineering

journal homepage: http://www.keaipublishing.com/en/journals/international-journal-of-cognitive-computing-in-engineering/

A DistilBERT-based hierarchical text classification for traffic analysis

Quang Tran Minh *, Do Thanh Thai

Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), 268 Ly Thuong Kiet, Dien Hong Ward, Ho Chi Minh City, Viet Nam
Vietnam National University Ho Chi Minh City (VNU-HCM), Linh Xuan Ward, Ho Chi Minh City, Viet Nam

ARTICLE INFO

Keywords:
Hierarchical multilabel text classification (HMTC)
Hierarchical text classification
Transfer learning
Traffic condition classification
ITS

ABSTRACT

Hierarchical multilabel text classification (HMTC) is constrained by the relationships between labels, often represented by nontrivial data structures such as directed acyclic graphs (DAGs). HMTC is widely utilized in real-world applications such as e-commerce and traffic condition classification for the intelligent transportation systems (ITS). However, the scarcity of training datasets and the strict constraints between labels make the HMTC one of the most challenging text classification problems. Recent advances in language modeling and transfer learning have enabled breakthroughs such as the pretraining of large-scale transformer models, vastly improving inference performance while lowering training requirements for task-specific models. This paper proposes a novel model, namely the DistilBERT branching hierarchical classification network (DB-BHCN), to maximize the utilization of advanced language comprehension capabilities in the DistilBERT. We also propose a combination with the adjacency wrapping matrix (AWX) layer, DB-BHCN+AWX, which is capable of full hierarchical compliance. We perform a comprehensive experimental analysis and find that both the proposed methods outperform the state-of-the-art Distil-BERT-based models. These results reveal the effectiveness and efficiency of the proposed mechanisms, showing their possibility to be applied in real-world applications, specifically in the ITS.

1. Introduction

Hierarchical multi-label text classification (HMTC), which classifies text using multiple labels organized as trees or directed acyclic graphs (DAGs), is widely applied in real-world applications such as medical coding (Peng et al., 2023), legal document categorization (Chalkidis et al., 2020), e-commerce product classification (Banerjee et al., 2022), and intelligent transportation systems (ITS) (Balasubramani et al., 2024; Garg & Kaur, 2023). In ITS, the target application of this work, HMTC helps organize complex traffic data. For example, traffic conditions are structured hierarchically, where the classifier identifies specific labels along with ancestor categories to preserve hierarchy (Silla, 2011), supporting public safety and urban traffic management. This enables pattern detection such as “traffic congestion → vehicle collision → car accident” or “traffic congestion → traffic flow → slow truck ahead” (Fig. 1), aiding enforcement and public policy (Zhang & Zhang, 2022; Zhou et al., 2021). Moreover, traffic data captured by cameras (Ramachandran & Sangaiah, 2021) can be described textually by large language models (Quang et al., 2024) and then classified by HMTC (Fig. 1) to provide structured information (see Fig. 2).

Conventional machine learning classifiers assume that target classes are mutually independent (Day & Khoshgoftaar, 2017). Hierarchical structures pose two challenges: data scarcity causing imbalanced and long-tail distributions (Chen et al., 2021), and hierarchical consistency requiring valid predictions along the hierarchy (Wehrmann et al., 2018). Flat classifiers fail to capture these dependencies (Silla, 2011) and early hierarchical models like top-down cascade models suffer error propagation (Kowsari et al., 2017). Deep learning improves expressiveness but struggles with small training datasets, leading to overfitting or underperformance (Keshari et al., 2020).

Transfer learning via pretrained models like BERT (Rai et al., 2022) enables knowledge reuse from large corpora (Devlin et al., 2019) and provides good performance on limited data. However, current BERT-based HMTC models often treat labels as a flat structure or apply post-hoc corrections without leveraging hierarchy during training (Ji et al., 2021). Therefore, a gap remains in integrating transfer learning with hierarchical reasoning in an end-to-end manner.

BERT’s bidirectional attention models word dependencies across contexts, which are crucial for text classification. Its transformer-based architecture captures semantic nuances, leading to state-of-the-art results in various NLP tasks (Devlin et al., 2019). Fine-tuning

* Corresponding author at: Vietnam National University Ho Chi Minh City (VNU-HCM), Linh Xuan Ward, Ho Chi Minh City, Viet Nam.
E-mail address: quangtran@hcmut.edu.vn (Q.T. Minh).

https://doi.org/10.1016/j.ijcce.2025.10.008
Received 5 October 2023; Received in revised form 22 September 2025; Accepted 18 October 2025
Available online 24 October 2025

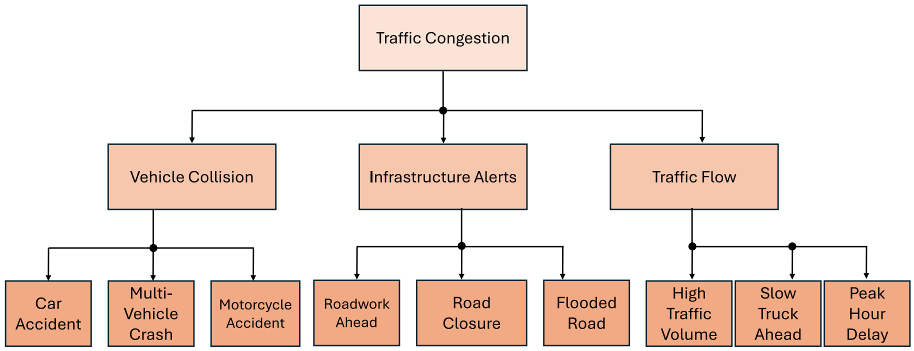

Fig. 1. Hierarchical multi-label text classification for traffic congestion.

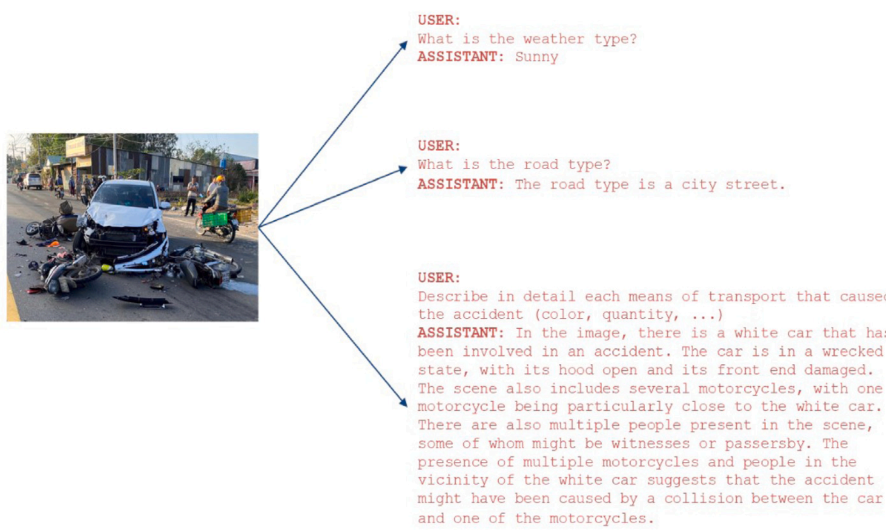

Fig. 2. A picture of a traffic accident is described by an LLM whose output description is then input to the HMTC for hierarchical traffic information classification.

BERT adds a shallow classification layer, benefiting low-resource domains. Domain-specific variants like BioBERT (Lee et al., 2020), SciBERT (Beltagy et al., 2019), ClinicalBERT (Huang et al., 2019), and Legal-BERT (Chalkidis et al., 2020) further enhance performance in specialized areas.

Despite their strengths, BERT and derivatives are not designed to model structured outputs like hierarchies. Therefore, their advantage in HMTC models is limited because they treat hierarchies as afterthoughts, applying structural corrections post-inference or using heuristic masking during training — ignoring hierarchical inductive biases.

Recent works integrate hierarchical reasoning more directly by augmenting BERT with graph-based structures such as graph convolutional networks (GCNs) or graph attention networks (GATs), embedding label hierarchies as graphs to propagate dependencies among related labels (He & Hu, 2025; Khairy et al., 2025; Wu et al., 2022; You et al., 2021). While effective, these methods are computationally heavy and struggle with cyclic or deep hierarchies. Hierarchical attention mechanisms dynamically weight parent and child labels based on context (Gupta et al., 2016), enabling contextually grounded and coherent predictions. However, these mechanisms require complex architectural changes and modifications, limiting full integration of structural constraints into loss functions.

Besides structure, leveraging semantic information in label descriptions is gaining attention. Many HMTC labels have textual definitions and BERT's language encoding enables models to treat classification as semantic matching of input and label descriptions (Chalkidis et al., 2020; Rios & Kavuluru, 2018). These approaches improve generalization to unseen or low-resource labels, but integrating hierarchy awareness into such approaches remains challenging.

Most BERT-based HMTC models lack end-to-end differentiability in enforcing structural constraints. Commonly, constraints like “if a child label is active, ancestors must be active” are applied post-hoc or via auxiliary terms that are not fully aligned with the main loss. A principled approach would define a loss that naturally enforces hierarchical consistency, for example, through constraint-based decoding (Ji et al., 2021) or structured prediction that treats label prediction as pathfinding.

In summary, although transfer learning with BERT has advanced NLP and text classification, its HMTC applications are limited due to the lack of native support for structured outputs and hierarchy-aware learning. Developing unified, scalable, end-to-end frameworks is an essential research direction. To address the aforementioned issues, we propose novel models that explicitly incorporate hierarchical label dependencies within a transfer learning framework, with main contributions summarized as follows:

- Propose a lightweight yet powerful model, namely the DistilBERT branching hierarchical classification network (DB-BHCN), that leverages DistilBERT, integrating transfer learning and hierarchical branching to improve classification accuracy.
- Enhance the DB-BHCN with adjacency wrapping matrix (DB-BHCN+AWX), where an AWX layer, a differentiable mechanism that enforces hierarchical constraints during the training phase, is introduced to avoid ad hoc corrections and ensure hierarchy-consistent prediction.

- The proposed models outperform state-of-the-art baselines on benchmark datasets. Concretely, DB-BHCN achieves substantial gains in F1-score and DB-BHCN+AWX reduces hierarchy violations by 94% with a negligible trade-off in other metrics.

By building on the generalization capabilities of pretrained transformers and introducing mechanisms for hierarchical compliance, our work demonstrates that effective HMTC is achievable even under the constraints of real-world data limitations and complex label structures, such as in ITS applications like traffic condition classification.

2. Related work

HMTC is a specialized multi-label text classification where classes are assigned following a hierarchical constraint. Classification may stop at internal classes without reaching leaves. Determining classification depth requires thresholds, posing an additional challenge (Fawzi et al., 2015; Toksöz & Ulusoy, 2017). This section reviews related work to position our proposed approaches to resolve the remaining difficulties in this research field.

2.1. Topological approaches

Various approaches have been proposed for hierarchical data classification, while adhering to hierarchical constraints can be broadly categorized as follows.

2.1.1. Leaf-level classification

The simplest hierarchical classification predicts only leaf-level classes and then infers parents by tracing back to the root. This is straightforward and efficient for hierarchies with consistent leaf labels but it ignores hierarchical structure, leading to suboptimal results when ground truth is at internal nodes. For example, in the ITS's traffic incident classification, a flat classifier may misclassify complex events like "multi-vehicle pile-up" if specific leaf labels are underrepresented. A hierarchy-aware model can correctly assign broader parent classes like "collision", improving performance when fine-grained labels are uncertain. Modeling subtype-parent relations also helps generalization and prevents incoherent predictions. Without hierarchical dependencies, models yield inconsistent or low-confidence outputs, especially for rare events or sparse data. Moreover, hierarchies with multiple parents consist of duplicate classes, increasing computational complexity when analyzing such data structures.

2.1.2. Internal node classification

The internal node classification approach (Kowsari et al., 2017) assigns a classifier to each internal node, predicting one of its child nodes and executing only classifiers along the predicted path. This leverages hierarchy by capturing parent–child relationships and provides meaningful predictions from the root to leaf nodes. For example, in traffic condition classification, the model first predicts "traffic congestion", then identifies whether that event occurred due to "vehicle collision" or "traffic flow", and further specifies subtypes like "car accident" is identified (see Fig. 1). However, this approach requires training many classifiers, increasing computational complexity and resources as the hierarchy grows. This approach also struggles with multiple-parent classes, which need node replication across paths, causing inefficiency.

2.1.3. Level-wise classification

The level-wise approach (Wehrmann et al., 2018) used in our model balances leaf-level and internal node methods by assigning one classifier per hierarchy level, often using the previous level's output as input. It captures hierarchical relations with simpler topology, avoids multiple-parent issues, and is more scalable than internal node classification. However, it requires post-processing to enforce hierarchy, adding complexity and reducing interpretability. Tuning multiple classifiers can also be resource-intensive for large, deep hierarchies.

This approach breaks the learning task into manageable sub-tasks per level, but it still needs post-processing to ensure consistency (e.g., classifying a "severe" incident under "traffic accident", not "weather event"). Its effectiveness depends on tuning each classifier well, which can increase computational costs, especially for large-scale ITS datasets.

2.1.4. Single classifier, multiple levels

The single classifier approach (Giunchiglia & Lukasiewicz, 2020) predicts all hierarchy levels at once, reducing computational overhead and simplifying architecture. It captures hierarchical dependencies directly, improving consistency across levels. For example, in the traffic flow classification, it predicts the entire path from "traffic flow" to "impact" in one step.

However, this method requires specialized loss functions to enforce hierarchical consistency, penalizing both incorrect and structurally inconsistent predictions. Post-processing is often needed to fix violations, complicating training processes and potentially hindering real-time performance, which is critical in ITS applications like dynamic traffic management.

Our proposed methods, namely the DB-BHCN and DB-BHCN+AWX, overcome the aforementioned limitations with an end-to-end differentiable framework. DB-BHCN uses DistilBERT and models hierarchical relations via a branching structure. On the other hand, DB-BHCN+AWX adds an AWX layer that enforces hierarchical consistency during the training process to eliminate post-hoc corrections. This improves accuracy, reduces hierarchy violations by 94%, and enhances transparency and real-time applicability.

2.2. Specialization in text classification with DistilBERT-based models for HMTC

Most pretrained hierarchical classification models use standard, encoded datasets like yeast classification (Clare, 2003), medical image annotation (Dimitrovski et al., 2011), and hierarchically labeled text (Lewis et al., 2004). While effective for classification, pretrained models are limited in encoding methods, especially for complex text. Our approaches are different by directly processing raw text and improving encoding. Unlike prior work that fine-tunes BERT-based encoders with domain knowledge (Ji et al., 2021; Lai et al., 2020; Lee & Hsiang, 2019), we are the first to apply a BERT-family encoder to HMTC, enabling extraction of more task-relevant information and potentially better accuracy. DistilBERT-based models have gained attention for HMTC due to their efficiency and lighter architecture. DistilBERT retains much of BERT's performance with a lower computational cost, which is suitable for large-scale tasks. Studies show that DistilBERT achieves comparable results to BERT with fewer resources (Chen et al., 2020; You et al., 2021).

2.3. Hierarchical multi-label text classification models

This section analyzes the state-of-the-art HMTC models, which closely relate to our proposed approaches.

2.3.1. HMCN-F

The hierarchical multi-label classification network with feature fusion (HMCN-F) (Wehrmann et al., 2018) uses a level-wise approach with one classifier per hierarchy level, efficiently capturing hierarchical relations and avoiding multiple-parent duplication. However, it requires post-processing for hierarchical compliance, adding computational complexity and reducing the model's interpretability. Relying on separate classifiers per level can be resource-intensive for large, deep hierarchies (see Fig. 3).

HMCN-F explicitly leverages hierarchical dependencies at each level by decomposing the task into simpler sub-problems, improving accuracy for coarse categories. Separate classifiers enable modular optimization, enhancing robustness in structured hierarchies.

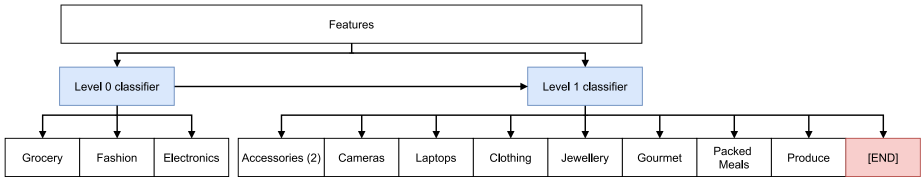

Fig. 3. A per-level classifier design, applied to HMCN-F.

Despite its strengths, HMCN-F limits information flow between levels, reducing prediction coherence in ambiguous cases. Its fixed hierarchy design makes it less adaptable to dynamic or incomplete taxonomies, limiting generalization where hierarchies evolve or are partially known. Performance depends heavily on the hierarchy's quality and stability.

2.3.2. C-HMCNN

The coherent hierarchical multi-label classification neural network (C-HMCNN) (Giunchiglia & Lukasiewicz, 2020) uses a single classifier to predict all hierarchy levels simultaneously, enabling end-to-end training and implicit cross-level feature learning. This compact structure suits resource-constrained and supports real-time applications. However, it requires specialized loss functions and post-processing, complicating the training process and deployment. Its lack of explicit hierarchical modeling reduces interpretability and limits use in transparency-critical settings.

Unlike HMCN-F and C-HMCNN, our proposed DB-BHCN and DB-BHCN+AWX offer more integrated, scalable solutions. DB-BHCN addresses computational complexity and limited inter-level sharing by combining a lightweight DistilBERT encoder with a branching architecture that mirrors label hierarchies. DB-BHCN+AWX further advances the aforementioned approach by embedding hierarchical constraints directly into the training process via a differentiable AWX layer, removing the need for post-processing while enhancing transparency. Experiments show that DB-BHCN improves F1-score and DB-BHCN+AWX cuts hierarchy violations by 94%, outperforming both HMCN-F and C-HMCNN on the benchmark datasets.

3. The proposed DB-BHCN model

This section presents in detail our proposed DB-BHCN model.

3.1. Design features

Through examining and pairing current state-of-the-art models with DistilBERT (Sanh et al., 2019), a compact but feature-complete version of BERT (Devlin et al., 2019) with almost equal performance, we concluded that BERT transfer learning works appropriately with shallower classification models. As the depth of a classifier increases, its capacity to effectively leverage knowledge from transfer learning diminishes, leading to a greater proportion of the network being initialized randomly and requiring training from scratch. To capture the effectiveness of a simple topology with DistilBERT for HMTC, we propose DB-BHCN whose overall computation graph is illustrated in Fig. 4. Its depth matches the hierarchy with no extra layers, while the output size at each layer corresponds to the number of classes at that level. Outputs are taken residually from each layer, branching from the main flow. Between layers, the hidden output passes through a nonlinear function, layer normalization, and dropout with a tunable probability.

Formally, let $x \in \mathbb{R}^{768\times 1}$ be our feature vector from the fine-tuned DistilBERT. Let H be the depth-padded hierarchy with depth $|H|$ and classes C_H, where C_{H_h} represents classes at level h. Within the model, let W and b be weights and biases with $W_h \in \mathbb{R}^{|C_{H_h}|\times|A^{h-1}|}$ and $b_h \in \mathbb{R}^{|A^{h-1}|\times 1}$ for layer h; $z^h \in \mathbb{R}^{|C_{H_h}|\times 1}$ is the linear output, A^h the hidden nonlinear output, and P^h the output scores at level h. Additionally, ϕ is a hidden nonlinear activation function (e.g., ReLU or tanh) and ψ is the output nonlinear activation (e.g., LogSoftmax in this work).

DB-BHCN's data flow is relatively straightforward. The input text is first encoded by a fine-tuned DistilBERT instance into x. This enters the first fully-connected level, producing z_1, which is then transformed by ϕ and ψ into A_1 and P_1, respectively:

$$z^1 = W^1 x + b^1 \tag{1a}$$

$$A^1 = \phi(z^1) \tag{1b}$$

$$P^1 = \psi(z^1) \tag{1c}$$

Then, for each non-leaf layer $0 < h \neq |H|$, the same happens with a small difference from the input layer, where the previous output is also concatenated with the original feature vector:

$$z^h = W^h(x \mathbin{+\!\!+} A^{h-1}) + b^h \tag{2a}$$

$$A^h = \phi(z^h) \tag{2b}$$

$$P^h = \psi(z^h) \tag{2c}$$

where $W^h \in \mathbb{R}^{|C_{H_h}|\times(|x|+|A^{h-1}|)}$. The last layer (where $h = |H|$) does not produce $A^{|H|}$ as there is no further layer after it. Layer normalization and dropout are performed in order but omitted here for clarity. Each hierarchical level's label indexing starts from zero, and the hierarchical label is represented as an ordered list of indices. The complete classification is the concatenation of one-hot vectors from each level.

3.2. Loss functions & training

Hierarchical classification has two conflicting requirements: accurate predictions at every hierarchical level so the entire path is correct and strict hierarchical compliance, where a wrong class at any level forces the rest of the path to stay within its descendants, unless it shares children with the ground truth. A hierarchically compliant model must predict the full path correctly and enforce compliance, even at the cost of accuracy. To satisfy both, we use two loss functions: a per-level loss L_L and a hierarchical loss L_H, both based on minibatch-averaged negative log-likelihood. Formally, let the formula be $\upsilon(y, f(x))$, with the local loss shown in Eq. (3).

$$L_L(y, f(x)) = \sum_{h=1}^{|H|} \upsilon(y_h, f(x)_h) \times c_L^h \tag{3}$$

where c_L^h is a per-level scaling factor for level h, computed as a symmetric linear interpolation between $-\gamma_L$ and γ_L, which is our level bias term. Its computation is presented in Eq. (4).

$$c_L^h = \frac{2\gamma_L}{|H|} h - \gamma_L - 1 \quad \forall h \in \mathbb{N} \text{ and } 1 \le h \le |H| \tag{4}$$

If $|H|$ is odd, then $c_L^{\lceil |H|/2 \rceil} = 0$. This local loss function tries to satisfy the first requirement by guiding the model to the ground truth on all levels at the same time.

Meanwhile, the hierarchical loss function is defined as:

$$L_H(f(x)) = \sum_{h=1}^{|H|-1} \upsilon(\text{parent}(h+1, \text{argmax}(f(x)_{h+1})), \quad f(x)_h) \times c_H^h \tag{5}$$

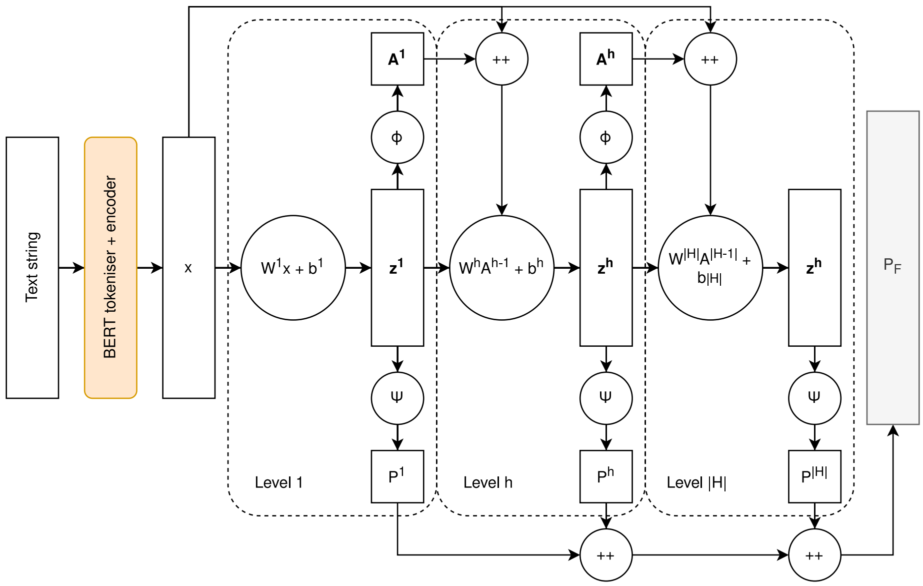

Fig. 4. Overall computation graph of the DB-BHCN model.

where $c_H^h \in \mathbb{R}^{|N_B|*1}$ is the hierarchical stabilization factor for level h, computed as the distance between 1 (the binary ground truth) and the locally-normalized score of the parent of the prediction at level $h+1$, and N_B is the batch size. A mathematical representation of c_H^h for a single training example is given in Eq. (6).

$$c_H^h = 1 - \frac{\text{parent}(h+1, \text{argmax}(P_F^{h+1})) - \min(P_F^{h+1})}{\max(P_F^{h+1}) - \min(P_F^{h+1}))}, \quad c_H^h \in [0,1] \forall h \in [1, |H|] \tag{6}$$

The hierarchical loss ensures compliance from the root down the tree, enforcing parent–child pairs to be compliant even if the parent prediction is wrong. However, using this loss with the local loss may confuse the training model. To address this, we introduce c_H, which reduces the hierarchical loss impact as predictions deviate more from targets.

The parent(i,j) function is implemented as a jagged 2D lookup vector from Algorithm 1, where index (i,j) corresponds to the jth class of level i (or $\sum_{u=1}^{i} |C_{H_u}| + j$ globally), and its value k is the parent node of class (i,j) at level $i-1$.

Algorithm 1: The *parent*() lookup vector algorithm.

Input: list of paths P, path depth d
Initialise $parent$:= vector();
for $i = 1$ **to** d **do**
  $parent$.append(map());
end for
for $i = 1$ **to** $|P|$ **do**
  $parent[0][L[0]] := P[0]$;
  for $j = 2$ **to** d **do**
    $childIdx := P[i][j]$;
    $parentIdx := P[i][j-1]$;
    $parent[j][childIdx] := parentIdx$;
  end for
end for

The final loss function is then the sum of the two loss functions, each scaled by a parameter λ_L and λ_H for the local and hierarchical loss functions, respectively, to vary their significance in the training. The proposed DB-BHCN model explicitly incorporates hierarchical label dependencies through a branching architecture where each level of the hierarchy is modeled as a separate output layer. The output of each layer not only predicts the classes at that level but is also concatenated with the original DistilBERT embedding and passed as input to the next layer. This enables the model to retain hierarchical context across levels and ensures that predictions at deeper levels are conditioned on higher-level decisions. Furthermore, the model uses a custom hierarchical loss function that penalizes violations of parent–child dependencies by ensuring that predicted child labels are supported by their predicted parent labels. This encourages the model to produce hierarchy-consistent predictions during training in an end-to-end differentiable manner.

4. The proposed DB-BHCN+AWX

This section presents in detail the proposed DB-BHCN+AWX, an improvement of DB-BHCN by combining it with the AWX layer. The AWX layer enforces hierarchical consistency by applying a fixed binary matrix to the model's leaf predictions, ensuring that a class is only activated if one of its descendants is confidently predicted. This fully differentiable mechanism integrates into the end-to-end training process, eliminating the need for post-processing while guiding structurally valid outputs.

4.1. Design features

One weakness of DB-BHCN is that it aims to minimize Eq. (5) but cannot always guarantee hierarchical compliance. To address this, we propose DB-BHCN+AWX, which incorporates the Adjacency Wrapping Matrix (AWX) layer (Masera & Blanzieri, 2018). The local outputs focus on loss minimization, while the hierarchical structure is enforced by transforming the last hidden layer's outputs. Specifically, these outputs are passed to the last hidden layer to produce leaf-level scores, which are then processed by AWX into hierarchically compliant predictions via matrix R. Matrix R records leaf node ancestry, with $R_{i,j} = 1$ if there is a directed path from the ith leaf class to the jth overall class; notably, $R_{i,j} = 1$ if $j = i + \sum_{h=1}^{|H|-1} |C_{H_h}|$, meaning the leaf class can always reach itself.

4.2. Implementing AWX for minibatched training

AWX has two variations: an ideal version using *max*(), and a more realistic, backpropagation-friendly version using l-norms. Let $z^{|H|}$ be the output of the final hidden layer in DB-BHCN, $|C_H|$ the total classes, f the Sigmoid activation, and $P_F \in \mathbb{R}^{|C_H|\times 1}$ the final prediction with P_{F_i} as the score for class i. Matrix $R \in \mathbb{R}^{|C_{H_{|H|}}|\times|C_H|}$ records leaf ancestry, where $R_{i,j} = 1$ if class j is an ancestor of class $i + |C_H| - |C_{H_{|H|}}|$. Vector

r_i is the ith column of R. Thus, R encodes leaf class ancestry, with leaf indices starting from zero and excluding non-leaf classes. The ideal version of AWX is defined as:

$$P_{F_i} = \max(r_i \otimes f(z^{|H|})) \tag{7}$$

and the ℓ-norm approximations are defined as:

$$P_{F_i} = \begin{cases} \|r_i \otimes f(z^{|H|})\|_\ell & \text{if} < 1 \\ 1 & \text{otherwise} \end{cases} \tag{8}$$

where, $\otimes$s indicates Hadamard products (Styan, 1973). These definitions apply to one example, but minibatch training uses $z^{|H|} \in \mathbb{R}^{B\times|C_{H|H|}|}$ with batch size B. The original definition is incompatible unless $B = 1$. We redefine P_F as $\mathbb{R}^{B\times|C_H|\times 1}$, and duplicate R B times into $R_B \in \mathbb{R}^{B\times|C_{H_{|H|}}|\times|C_H|}$. Similarly, $z^{|H|}$ is stacked $|C_H|$ times to form $z_B^{|H|} \in \mathbb{R}^{B\times|C_{H_{|H|}}|\times|C_H|}$. This allows each class score in P_F to have its own mask based on $z^{|H|}$ and r_i. Substituting these into Eqs. (7) and (8) extends AWX for minibatch training without loops as shown in Eq. (9) and (10).

$$P_F = max(R_B \otimes f(z_B^{|H|})) \tag{9}$$

$$P_F = \begin{cases} \|R_B \otimes f(z_B^{|H|})\|_\ell & \text{if} < 1 \\ 1 & \text{otherwise} \end{cases} \tag{10}$$

The representation of the hierarchical structure for use by the AWX layer is achieved through the matrix R, which is generated from training data using a procedure that encodes the path-based hierarchy of classes into a binary matrix format.

4.3. Integrating AWX with DB-BHCN

DB-BHCN requires minor changes to integrate with AWX, mainly omitting the hierarchical loss since AWX uses R which does not need to learn hierarchical constraints via backpropagation. We found that the hierarchical loss conflicts with AWX, so we replace it with a binary cross-entropy (BCE) loss from the AWX output, combined with the local loss, which helps this variant quickly improve in the training process. Additionally, Sigmoid activation is applied before feeding the last hidden layer to AWX, while LogSoftmax is still used on concatenated local outputs for the existing local loss.

5. Experimental analysis

This section analyzes the effectiveness and the efficiency, specifically the response times to multiple requests, of the proposed models (DB-BHCN and DB-BHCN+AWX) compared to the state-of-the-art studies, namely TF-IDF+HSGD, DB+AHCMN-F, and DB+AC-HMCNN upon experimental evaluations. In addition, we present ablation studies to highlight the effects of important design elements in the proposed models.

5.1. Datasets and environment setting

To evaluate the proposed models, six public datasets with product information and category paths from two large e-commerce platforms were used. One use case is providing categorization suggestions based on product names; another is delivering hierarchical traffic information (e.g., "congestion" → "incidence" → "car accident") in response to user queries.

Five datasets are from Amazon, provided by UC San Diego (He & McAuley, 2016; McAuley et al., 2015), representing different top-level categories. The sixth is a small Walmart product dataset,[1] a subset of a larger collection. Table 1 summarizes datasets' characteristics: C is the set of classes, C_h the subset at the hierarchy level h, $\omega(C_h)$ the average examples per class, and N_{train}, N_{val}, N_{test} the training, validation, and test set sizes. These datasets are hierarchically labeled product taxonomies representing real-world multi-level classifications. Therefore, they are well-suited for benchmarking HMTC models like our proposed DB-BHCN and DB-BHCN+AWX ones. While these datasets originate from e-commerce, they possess key structural characteristics such as multi-level label hierarchies, label imbalance, and long-tail distributions, that are also present in the ITS applications. For example, both domains require accurate categorization across abstract and fine-grained levels (e.g.,"traffic congestion → incident → car accident"), and hierarchical consistency is critical for meaningful outputs. Therefore, the proposed models can be utilized to classify textual descriptions of traffic scenes (e.g., from LLMs or sensors) into hierarchical traffic categories efficiently.

All five models, TF-IDF+HSGD, DB+AHCMN-F, DB+AC-HMCNN, and the proposed DB-BHCN and DB-BHCN+AWX, were trained for five epochs on each dataset. Each dataset was split into training, validation, and test sets in an 8:1:1 ratio using fixed-seed random sampling. The non-deep-learning model trained up to 1000 iterations with early stopping. For deep-learning models, minibatch order was randomized between training and validation. Experiments ran on commodity hardware. Each DistilBERT-based model used different hyperparameters based on its topology. The training process employed the Adam optimizer for DistilBERT models and default SGD for others.

5.2. Effectiveness

In order to evaluate the effectiveness of the proposed methods, accuracy and $AU(\overline{PRC})$ are concurrently utilized. Accuracy offers a glance of real-world performance (but is greatly affected by how results are parsed) while $AU(\overline{PRC})$ is a standard, universal metric for evaluating a classification model (Qi et al., 2021). $AU(\overline{PRC})$ stands for *area under precision–recall (PR)* curve, whose points $(\overline{Prec}, \overline{Rec})$ are computed in Eqs. (11) and (12) (Giunchiglia & Lukasiewicz, 2020).

$$\overline{Prec} = \frac{\sum_{i=1}^{n} \text{TP}_i}{\sum_{i=1}^{n} \text{TP}_i + \sum_{i=1}^{n} \text{FP}_i} \tag{11}$$

$$\overline{Rec} = \frac{\sum_{i=1}^{n} \text{TP}_i}{\sum_{i=1}^{n} \text{TP}_i + \sum_{i=1}^{n} \text{FN}_i} \tag{12}$$

where $\overline{Prec}, \overline{Rec}$ represent precision and recall values, while TP_i, FP_i, and FN_i are the number of true positives, false positives, and false negatives for class i, respectively.

The two state-of-the-art generic hierarchical classification models are HMCN-F (Wehrmann et al., 2018) and C-HMCNN (Giunchiglia & Lukasiewicz, 2020). C-HMCNN's code is publicly available and used for evaluation, while HMCN-F was implemented from scratch. Both replaced batch normalization with NLP-friendly alternatives (Shen et al., 2020) and use DistilBERT embeddings (768 values). DistilBERT finetuning was disabled for HMCN-F due to catastrophic forgetting. As a baseline, we constructed TF-IDF+HSGD, a CPU-based classical model using TF-IDF (Rajaraman & Ullman, 2011) and SGD with modified Huber loss (Zhang, 2004) for each internal DAG node, producing leaf predictions. These three models are compared with the proposed ones.

Each model ran five times on six datasets (Table 1). Averaged $AU(\overline{PRC})$ and accuracy are reported in Tables 2 and 3. Test outputs include scores (e.g., global P_F for HMCN-F, per-level outputs for DB-BHCN, and AWX output for DB-BHCN+AWX) and targets for direct comparison. Predictions use model-specific functions relying on argmax per level. Both metrics are measured at the leaf level, the most challenging with the fewest examples per class, highlighting each model's ability to leverage hierarchical clues for better leaf-level decisions.

Regarding $AU(\overline{PRC})$ (Table 2), we observe that the first proposed model, DB-BHCN, consistently outperforms all baselines across all datasets, achieving the highest model average of 0.820 and the best

[1] https://www.kaggle.com/promptcloud/walmart-product-data-2019

Table 1
Characteristics of the datasets used to benchmark the proposed models.

ID	Dataset	Attribute	$\|C\|$	N_{train}	N_{val}	N_{test}	$\omega(C_0)$	$\omega(C_1)$
1	Amazon Arts Crafts and Sewing	64	115	174 176	21 772	21 772	19 792.7	2093.46
2	Amazon Musical Instruments	64	74	81 278	10 160	10 160	6773.2	1722
3	Amazon Grocery and Gourmet Food	64	279	188 749	23 593	23 594	9074.46	932.553
4	Amazon Industrial and Scientific	64	383	105 446	13 181	13 181	4881.78	370.247
5	Amazon Electronics	64	140	481 622	60 202	60 203	43 001.9	4777.99
6	Walmart	32	324	23 365	2920	2921	942.129	99.6792

Table 2
The $AU(\overline{PRC})$ of the five models over the leaf level obtained on the test set.

Source	Dataset	TF-IDF+HSGD	DB+AHMCN-F	DB+AC-HMCNN	DB-BHCN (proposed)	DB-BHCN+AWX (proposed)
Amazon	Arts_Crafts	0.457	0.699	0.694	**0.831**	0.815
	Electronics	0.413	0.732	0.656	**0.812**	0.792
	Grocery	0.322	0.611	0.566	**0.770**	0.746
	Industrial_and_Scientific	0.537	0.686	0.659	**0.848**	0.827
	Musical_Instruments	0.473	0.729	0.759	**0.841**	0.822
Walmart	30k	0.314	0.729	0.758	**0.818**	0.814
	Model average	0.419	0.698	0.682	**0.820**	0.803
	Average ranking	5.000	3.333	3.667	1.000	2.000

Table 3
Comparison between the five models on raw accuracy over the leaf level obtained on the test set.

Source	Dataset	TF-IDF+HSGD	DB+AHMCN-F	DB+AC-HMCNN	DB-BHCN (proposed)	DB-BHCN+AWX (proposed)
Amazon	Arts_Crafts	0.817	0.600	0.497	0.868	**0.870**
	Electronics	0.737	0.785	0.677	0.860	**0.864**
	Grocery	0.571	0.650	0.730	0.803	**0.811**
	Industrial_and_Scientific	0.781	0.653	0.666	0.844	**0.880**
	Musical_Instruments	0.783	0.675	0.694	0.880	**0.887**
Walmart	30k	0.613	0.559	0.711	0.761	**0.781**
	Model average	0.717	0.654	0.662	0.836	**0.849**
	Average ranking	3.667	4.333	4.000	2.000	1.000

average ranking of 1.000. The second-best model, DB-BHCN+AWX (an adaptation of DB-BHCN with an AWX module), also performs strongly with an average score of 0.803, ranking second overall. Interestingly, DB+AHMCN-F emerges as the third best model, achieving a mean $AU(\overline{PRC})$ of 0.698. Despite its simpler structure, this model slightly outperforms DB+AC-HMCNN (average 0.682), indicating that the modifications introduced in AHMCN-F may offer better compatibility with the DistilBERT features. The original C-HMCNN model, despite being evaluated only on the specialized datasets mentioned in Section 2.2, demonstrates better performance than both HMCN variants. However, when DistilBERT fine-tuning is applied to it, performance unexpectedly degrades. Further experiments confirmed that fine-tuning the encoder alone did improve feature representations, suggesting that the post-processing mechanism used in C-HMCNN is not well-suited to the characteristics of DistilBERT-encoded features.

Overall, Table 2 shows that our proposed models (DB-BHCN and DB-BHCN+AWX) achieve the highest performance across all datasets. The table also reports both the mean $AU(\overline{PRC})$ and the average ranking across models, offering a comprehensive comparison. We highlight that DB-BHCN ranks first on every dataset, validating the robustness and generalizability of our approach.

According to accuracy (Table 3), we find that DB-BHCN+AWX consistently achieves the highest performance across all datasets, with a model average of 84.9% and the best average ranking of 1.000. This result highlights the effect of enforcing strict hierarchy compliance through the AWX module. In particular, DB-BHCN+AWX shows consistent gains over DB-BHCN, which only utilizes a hierarchical loss function. For example, on the challenging Walmart_30k dataset, DB-BHCN+AWX achieves an accuracy of 78.1%, compared to 76.1% for DB-BHCN, and significantly higher than the remaining baselines. This confirms the robustness of our AWX-enhanced architecture under both domain-specific and large-scale classification settings. However, this improvement in accuracy comes at a cost in terms of $AU(\overline{PRC})$, which better reflects a model's ability to separate true labels from false ones. As shown in Table 2, the vanilla DB-BHCN achieves a higher $AU(\overline{PRC})$ than its AWX-enhanced counterpart on every dataset. This trade-off suggests that while AWX improves label compliance, it may impose constraints that limit probabilistic separability in some cases. To verify the statistical significance of the performance differences, we conducted

Table 4
API response time by percentile.

Percentile	50%	66%	75%	80%	90%	95%	98%	99%	99.9%	100%
Latency (ms)	140	150	150	160	170	180	190	390	400	400

Table 5
Path-average accuracy, leaf-level accuracy and leaf-level $AU(\overline{PRC})$ metrics of four different ablation configurations. The bottom-right configuration is the normal, non-ablated one used for Section 5.2.

Path-average accuracy	$\lambda_H = 0$	$\lambda_H = 0.7$
w/o DFI	0.671	0.655
w/ DFI	0.721	0.728
Leaf-level accuracy	$\lambda_H = 0$	$\lambda_H = 0.7$
w/o DFI	0.538	0.520
w/ DFI	0.639	0.650
Leaf-level $AU(\overline{PRC})$	$\lambda_H = 0$	$\lambda_H = 0.7$
w/o DFI	0.919	0.912
w/ DFI	0.943	0.942

a Friedman test on the $AU(\overline{PRC})$ results across all datasets. The test yielded a p-value of 1.306×10^{-4}, which is below the 0.05 threshold, indicating that the observed performance differences among models are statistically significant.

5.3. Efficiency

To validate computational performance, we measured response time and throughput under concurrent user requests on commodity hardware with an NVIDIA GeForce GTX 1050 Mobile (4 GB). The experiment simulated up to 10 concurrent users sending API requests at maximum speed, ramping up by adding one user per second. Each user waits for their own response before sending the next request, independent of other users.

This simulates a high-stress scenario like ITS, where many users report and query traffic conditions hierarchically (e.g., "congestion" → "incidence" → "car accident") for routing decisions. Requests (25 bytes each) were sent concurrently to test queuing and resource utilization. Since users and services share the same host, network delays are negligible; here we focus mainly on computational performance.

Several one-minute executions have been conducted, and we recorded an average of **4506** completed requests, about 75.1 requests per second, with average, minimum, and maximum latencies of around 124.77 ms, 59.72 ms, and 1050.75 ms, respectively. The large max latency occurred during initial ramp-up as the service loaded model artifacts, with subsequent latencies stabilizing closer to the average.

Table 4 provides a detailed breakdown of API response times across different percentiles, offering a clearer understanding of the system's latency distribution. We can observe that 90% of API responses are completed within 170 ms, while 99% are returned in less than 390 ms. Furthermore, the maximum recorded latency is capped at 400 ms, even at the 100th percentile.

These results indicate that the system exhibits low-latency behavior with tight distribution bounds, making it suitable for real-time deployment scenarios. Even under high-load or edge cases, the response times remain well below thresholds commonly expected in practical applications. This is particularly important for time-sensitive use cases such as ITS applications, where timely predictions and responses are critical.

5.4. Ablation study

To gain a deeper insight into the design of DB-BHCN, we conducted an ablation study focusing on two key architectural components: the hierarchical loss function and the direct feature inputs (DFI) from the DistilBERT encoder into each classification layer. Because both components are directly related to the exploitation of hierarchical structure, we evaluated the configurations using the *path-average accuracy* metric, which averages prediction accuracy across all levels of the hierarchy. All experiments were conducted on a fixed 10% subset of the Walmart_30k dataset and repeated 10 times to ensure statistical robustness. The results are reported in Table 5, which includes three evaluation metrics: path-average accuracy, leaf-level accuracy, and $AU(\overline{PRC})$.

The results show that enabling DFI consistently improves performance across all metrics. For example, when $\lambda_H = 0.7$, enabling DFI increases path-average accuracy from 0.655 to 0.728 and improves leaf-level accuracy from 0.520 to 0.650. This suggests that providing the original DistilBERT features directly to each layer helps downstream classifiers better leverage pretrained representations. Meanwhile, the hierarchical loss ($\lambda_H > 0$) also shows positive effects on accuracy metrics, especially when DFI is enabled. However, we observe a slight decrease in $AU(\overline{PRC})$ with hierarchical loss enabled (e.g., 0.943 → 0.942), indicating a minor trade-off. This may be due to the hierarchical loss occasionally steering optimization toward more globally coherent predictions at the expense of maximizing local discriminative power. Overall, the best-performing configuration corresponds to the bottom-right cell of Table 5, which combines both DFI and hierarchical loss ($\lambda_H = 0.7$) resulting AU(PRC) of 0.942, this is the standard setting used throughout Section 5.2.

5.5. Discussion

The proposed models effectively address challenges of sparse and imbalanced data in HMTC, particularly in real-world domains like the ITS. First, by leveraging transfer learning via DistilBERT, the models benefit from pretrained semantic representations, improving generalization for underrepresented classes while significantly reducing model size and inference time, enabling deployment on low-end hardware (e.g., GTX 1050 Mobile GPU).

Second, the DB-BHCN decomposes the classification task into level-specific subtasks, reducing label sparsity and enhancing learning for deep-level, low-frequency classes. Third, a novel hierarchical loss function enforces consistency across label levels and facilitates learning from limited or ambiguous data. Additionally, the AWX layer in the DB-BHCN+AWX variant imposes explicit hierarchical constraints during prediction, improving robustness against noise and imbalance with minimal computational overhead. It can even replace hierarchical loss without requiring complex backpropagation.

Experimental results on long-tail datasets, including Walmart and Industrial, demonstrate significant model performance. DB-BHCN

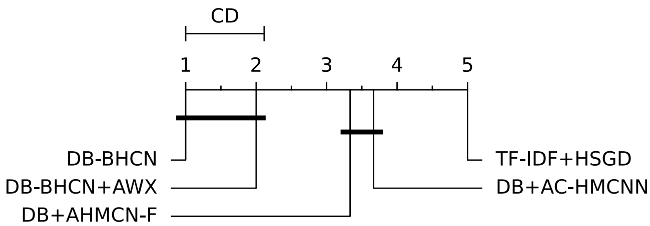

Fig. 5. Leaf-level critical difference diagrams for the Friedman–Nemenyi post-hoc test in $AU(\overline{PRC})$.

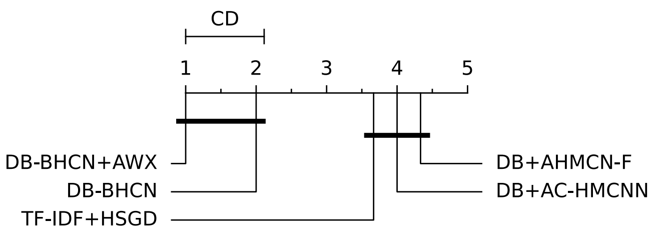

Fig. 6. Leaf-level critical difference diagrams for the Friedman–Nemenyi post-hoc test in accuracy.

achieves the best $AU(\overline{PRC})$ score, indicating superior label ranking capability. DB-BHCN+AWX, while trading off a small portion of $AU(\overline{PRC})$, achieves higher accuracy (84.9%) and reduces hierarchical violations by 94%, making it more suitable for real-world ITS deployment where structural validity is critical. Under high-load simulations, it sustains real-time performance, with 90% of requests processed in under 170 ms.

To validate these findings, we conducted statistical significance testing using the Friedman test followed by a Nemenyi post-hoc test. The resulting critical distance diagrams (Figs. 5 and 6, following Demšar, 2006) show that both proposed models significantly outperform all baseline methods across datasets and metrics. The performance difference between DB-BHCN and DB-BHCN+AWX, however, is statistically insignificant, suggesting that the choice between them can be guided by application-specific constraints such as the need for strict hierarchy compliance or latency requirements.

6. Conclusions

This paper proposed two novel DB-BHCN-based models that effectively integrate the advantages of transfer learning from DistilBERT and the hierarchical nature of multilabel data for hierarchical text classification tasks. Two variants of the proposed approach have been devised: (i) A novel hierarchical-loss-enabled learning model that learns features within hierarchical constraints by means of a hierarchical loss function — the DB-BHCN, and (ii) A mathematically constrained variant that employs an extended version of the AWX layer — the DB-BHCN+AWX.

Thorough experimental results on six publicly available datasets show that the proposed approaches outperform state-of-the-art methods in both effectiveness and efficiency. These results also reveal the practical applicability of the proposed approaches, especially in the field of traffic condition classification, where the data are often organized into multiple levels of abstraction, i.e., multilabel classification. Concretely, the proposed methods can be applied to ITS applications, where traffic conditions are classified into different levels of detail such as “congestion” → “incident” → “car accident”, thus providing high-granularity information to users.

However, there are still several limitations that need to be further investigated and addressed to enhance the performance of the proposed methods. Designing an appropriate data structure to handle hierarchical data and optimizing search algorithms in machine learning models should be thoroughly investigated. Moreover, to analyze hierarchical data efficiently, hierarchy-aware or graph-based machine learning models such as graph neural networks (GNNs) should be explored. In addition, the potential benefits of large language models (LLMs) such as GPT-4, Gemini, and LLaMA over BERT in hierarchical multilabel classification should be further studied.

In the future, we intend to further develop the proposed approaches to enhance inter-layer knowledge extraction and data flow, perform further experiments on more traffic condition datasets, and bring the proposed mechanisms into real-world applications, especially in smart city infrastructures and ITS services.

CRediT authorship contribution statement

Quang Tran Minh: Writing – review & editing, Writing – original draft, Validation, Supervision, Resources, Project administration, Methodology, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization. **Do Thanh Thai:** Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Formal analysis, Data curation, Conceptualization.

Declaration of competing interest

The authors declare that there are no conflicts of interest regarding the publication of this paper.

Acknowledgments

This research is funded by Vietnam National University Ho Chi Minh City (VNU-HCM) under grant number DS2024-20-01.

References

Balasubramani, S., Aravindhar, J., Renjith, P. N., & Ramesh, K. (2024). DDSS: Driver decision support system based on the driver behaviour prediction to avoid accidents in intelligent transport system. *International Journal of Cognitive Computing in Engineering*, *5*, 1–13.

Banerjee, A., Gupta, S., Bansal, V., & Jhala, P. (2022). E-commerce product classification using distributional semantics. In *The 6th int. conference on deep learning technologies* (pp. 904–912).

Beltagy, I., Lo, K., & Cohan, A. (2019). Scibert: A pre-trained language model for scientific text. In *The conference on empirical methods in natural language processing* (pp. 3606–3611).

Chalkidis, I., Fergadiotis, M., & Aletras, N. (2020). Legal-BERT: The legal domain pre-trained language model. In *The 58th annual meeting of the association for computational linguistics* (pp. 2890–2896).

Chen, B., Pan, X., & Luo, B. (2021). Research on power-law distribution of long-tail data and its application to tourism recommendation. *Industrial Management & Data Systems*, *121*(6), 1268–1286.

Chen, L., Wang, Y., & Li, J. (2020). A hierarchical classification approach to traffic incident detection using natural language processing. *IEEE Transactions on Intelligent Transportation Systems*, *21*(4), 1780–1790.

Clare, A. (2003). *Machine learning and data mining for yeast functional genomics* (Ph.D. thesis), Citeseer.

Day, O., & Khoshgoftaar, T. M. (2017). A survey on heterogeneous transfer learning. *Journal of Big Data*, 29.

Demšar, J. (2006). Statistical comparisons of classifiers over multiple data sets. *Journal of Machine Learning Research*, *7*, 11–16.

Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: pre-training of deep bidirectional transformers for language understanding. In J. Burstein, C. Doran, & T. Solorio (Eds.), *The Int. Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Minneapolis, MN, USA, June 2-7*. Association for Computational Linguistics.

Dimitrovski, I., Kocev, D., Loskovska, S., & Dzeroski, S. (2011). Hierarchical annotation of medical images. *Pattern Recognition*, 2436–2449.

Fawzi, A., Davies, M. E., & Frossard, P. (2015). Dictionary learning for fast classification based on soft-thresholding. *International Journal of Computer Vision*, 306–321.

Garg, T., & Kaur, G. (2023). A systematic review on intelligent transport systems. *Journal of Computational and Cognitive Engineering*, *2*(3), 175–188.

Giunchiglia, E., & Lukasiewicz, T. (2020). Coherent hierarchical multi-label classification networks. *Advances in Neural Information Processing Systems*, 9662–9673.

Gupta, V., Karnick, H., Bansal, A., & Jhala, P. (2016). Product classification in E-commerce using distributional semantics. URL https://arxiv.org/abs/1606.06083.

He, L., & Hu, X. (2025). The application of GCN algorithm in building construction knowledge graph updating under the combination of artificial intelligence and knowledge management. *International Journal of Cognitive Computing in Engineering*, *6*, 65–73.

He, R., & McAuley, J. J. (2016). Ups and downs: Modeling the visual evolution of fashion trends with one-class collaborative filtering. In J. Bourdeau, J. Hendler, R. Nkambou, I. Horrocks, & B. Y. Zhao (Eds.), *The 25th int. conference on world wide web, montreal, Canada, April 11 - 15* (pp. 507–517). ACM.

Huang, K., Altosaar, J., & Ranganath, R. (2019). Clinicalbert: Modeling clinical notes and predicting hospital readmission. arXiv preprint arXiv:1904.05342.

Ji, Y., Zhou, Z., Liu, H., & Davuluri, R. V. (2021). DNABERT: pre-trained bidirectional encoder representations from transformers model for DNA-language in genome. *Bioinformatics*, 2112–2120.

Keshari, R., Ghosh, S., Chhabra, S., Vatsa, M., & Singh, R. (2020). Unravelling small sample size problems in the deep learning world. In *6th IEEE int'l conference on multimedia big data, new Delhi, India, sep. 24-26* (pp. 134–143).

Khairy, M., Mokhtar, H. M., & Abdalla, M. (2025). Adaptive traffic prediction model using graph neural networks optimized by reinforcement learning. *International Journal of Cognitive Computing in Engineering*, *6*, 431–440.

Kowsari, K., Brown, D. E., Heidarysafa, M., Meimandi, K. J., Gerber, M. S., & Barnes, L. E. (2017). Hdltex: Hierarchical deep learning for text classification. In X. Chen, B. Luo, F. Luo, V. Palade, & M. A. Wani (Eds.), *16th IEEE int. conference on machine learning and applications, Mexico, dec. 18-21* (pp. 364–371).

Lai, T. M., Tran, Q. H., Bui, T., & Kihara, D. (2020). A simple but effective bert model for dialog state tracking on resource-limited systems. In *IEEE int'l conference on acoustics, speech and signal processing, Barcelona, Spain, May 4-8* (pp. 8034–8038).

Lee, J.-S., & Hsiang, J. (2019). PatentBERT: Patent classification with fine-tuning a pre-trained BERT model. In *ArXiv:1906.02124*, *Computing Research Repository (CoRR)*, In *ArXiv:1906.02124*, 1–6.arXiv:1906.02124,

Lee, J., Yoon, W., Kim, S., Kim, D., Kim, S., So, C. H., & Kang, J. (2020). Biobert: a pre-trained biomedical language representation model for biomedical text mining. *Bioinformatics*, *36*(4), 1234–1240.

Lewis, D. D., Yang, Y., Rose, T. G., & Li, F. (2004). RCV1: a new benchmark collection for text categorization research. *Journal of Machine Learning Research*, 361–397.

Masera, L., & Blanzieri, E. (2018). AWX: an integrated approach to hierarchical-multilabel classification. In M. Berlingerio, F. Bonchi, T. Gärtner, N. Hurley, & G. Ifrim (Eds.), *Machine learning and knowledge discovery in databases - European conference, Ireland, sep. 10-14*. Springer.

McAuley, J. J., Targett, C., Shi, Q., & van den Hengel, A. (2015). Image-based recommendations on styles and substitutes. In R. Baeza-Yates, M. Lalmas, A. Moffat, & B. A. Ribeiro-Neto (Eds.), *The 38th int. ACM SIGIR conference on research and development in information retrieval, santiago, Chile, aug. 9-13* (pp. 43–52).

Peng, R., Wang, Z., Wang, Y., Wang, Y., Zhang, F., Zhou, L., Yang, F., & Zhang, Z. (2023). Medical text classification based on the discriminative pre-training model and prompt-tuning. *Journal of Medical Internet Research*, *25*(6), Article e319213.

Qi, Q., Luo, Y., Xu, Z., Ji, S., & Yang, T. (2021). Stochastic optimization of area under precision-recall curve for deep learning with provable convergence. *Computing Research Repository (CoRR)*, 1–25, arXiv:2104.08736.

Quang, T. M., Phan, T. N., Duc, B. T., Thai, D. T., & Huu, P. N. (2024). Towards an approach of traffic information extraction through ChatGPT. In *Int. conference on intelligence of things* (pp. 45–54). Cham: Springer Nature Switzerland.

Rai, N., Kumar, D., Kaushik, N., Raj, C., & Ali, A. (2022). Fake news classification using transformer based enhanced LSTM and BERT. *International Journal of Cognitive Computing in Engineering*, *3*, 98–105.

Rajaraman, A., & Ullman, J. D. (2011). Data mining. In *Mining of massive datasets* (pp. 1–17). Stanford University.

Ramachandran, A., & Sangaiah, A. K. (2021). A review on object detection in unmanned aerial vehicle surveillance. *International Journal of Cognitive Computing in Engineering*.

Rios, A., & Kavuluru, R. (2018). Multi-task learning for medical text classification with neural networks. In *The conference on empirical methods in natural language processing* (pp. 3849–3858).

Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019). Distilbert, a distilled version of BERT: smaller, faster, cheaper and lighter. *Computing Research Repository (CoRR)*, 1–5, arXiv:1910.01108.

Shen, S., Yao, Z., Gholami, A., Mahoney, M. W., & Keutzer, K. (2020). Rethinking batch normalization in transformers. *Computing Research Repository (CoRR)*, 1–18, arXiv:2003.07845.

Silla, A. A. (2011). A survey of hierarchical classification across different application domains. *Data Mining and Knowledge Discovery*, 31–72.

Styan, G. P. (1973). Hadamard products and multivariate statistical analysis. *Linear Algebra and its Applications*, *6*, 217–240.

Toksöz, M. A., & Ulusoy, I. (2017). Hyperspectral image classification via kernel basic thresholding classifier. *IEEE Trans. Geosci. Remote. Sens.*, 715–728.

Wehrmann, J., Cerri, R., & Barros, R. (2018). Hierarchical multi-label classification networks. In J. Dy, & A. Krause (Eds.), *Proceedings of machine learning research*: *80*, *The 35th int. conference on machine learning* (pp. 5075–5084). PMLR.

Wu, Y., Li, X., & Zhou, J. (2022). A deep learning approach to text classification with pre-trained language models. *Journal of Machine Learning Research*, *23*(1), 1–30.

You, Y., Zhang, Z., & Wang, J. (2021). A comprehensive survey on pre-trained language models for natural language processing. *IEEE Access*, *9*, 101204–101225.

Zhang, T. (2004). Solving large scale linear prediction problems using stochastic gradient descent algorithms. In *The 21st int. conference on machine learning* (p. 116).

Zhang, H., & Zhang, Y. (2022). Hierarchical representation learning for intelligent transportation systems. *Transportation Research Part C: Emerging Technologies*, *138*, Article 103629.

Zhou, J., Liu, T., & Ma, J. (2021). An ontology-based approach for traffic violation classification and enforcement policy design. *Expert Systems with Applications*, *185*, Article 115605.